

01/09/2026

Level 2: Recursion Core

① Factorial using Recursion:

Problem: Given a number n , calculate its factorial.

Example: $5! = 5 \times 4 \times 3 \times 2 \times 1 = 120$ I/P: 5
O/P: 120

Recursive thinking: Instead of $5 \times 4 \times 3 \times 2 \times 1$ we can go with the factorial using a smaller factorial.

$$5! = 5 \times 4! , 4! = 4 \times 3! , 3! = 3 \times 2! , 2! = 2 \times 1!$$

$$\text{factorial}(n) = n \times \text{factorial}(n-1)$$

Base case: $1! = 1$, so we can use,

if $n == 0$ or $n == 1$:
return 1

Recursive case: return $n * \text{factorial}(n-1)$

Python Template:

```
def factorial(n):
```

```
    if n == 0 or n == 1:
```

```
        return 1
```

```
    return n * factorial(n-1)
```

Base case

$n == 0$ or $n == 1$

↓

return 1

otherwise

↓
 $n \times \text{fact}(n-1)$

Recursive vs loop:

```
def fact(n):
```

```
    if n <= 1:
```

```
        return 1
```

```
    return n * fact(n-1)
```

```
def fact(n):
```

```
    result = 1
```

```
    for i in range(1, n+1):
```

```
        result *= i
```

```
    return result
```

Both: T.C : $O(n)$

S-E : $O(n)$

② Fibonacci using Recursion

Problem: fibonacci sequence: 0, 1, 1, 2, 3, 5, 8, 13, 21, ...
Each number is the sum of the previous two

$$F(n) = F(n-1) + F(n-2) \quad \text{Standard definition:}$$

$$F(0) = 0$$

$$F(1) = 1$$

So,

$$\begin{aligned} F(2) &= F(1) + F(0) \\ &= 1 + 0 \\ &= 1 \end{aligned}$$

$$\begin{aligned} F(3) &= F(2) + F(1) \\ &= 1 + 1 \\ &= 2 \end{aligned}$$

$$\begin{aligned} F(4) &= F(3) + F(2) \\ &= 2 + 1 \\ &= 3 \end{aligned}$$

Recursion thinking:

In factorial:

$F(n) \rightarrow$ one smaller problem
 $n \times F(n-1)$

Fibonacci:

$$F(n)$$

$\downarrow$

Two smaller problems

$$F(n-1) + F(n-2)$$

$$fib(n) = fib(n-1) + fib(n-2)$$

Python template:

```
def fib(n):
```

```
    if n == 0:
        return 0
```

```
    if n == 1:
        return 1
```

```
    return fib(n-1) + fib(n-2)
```

$\rightarrow$ Iterative version

$$T.C: O(2^n)$$

$$S.C: O(n)$$

Iterative version:

```
def fib(n):
```

```
    a = 0
```

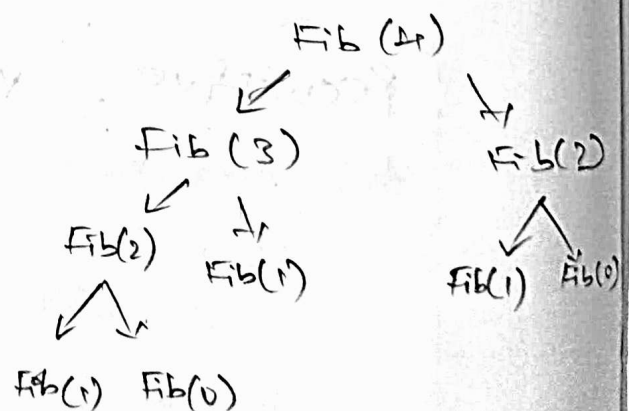
```
    b = 1
```

```
    for _ in range(n):
```

```
        a, b = b, a + b
```

```
    return a
```

Recursion Tree:



$$T.C: O(n)$$

$$S.C: O(1)$$

③ Sum of n numbers:

Problem: Given n, calculate $1 + 2 + 3 + \dots + n$

Example: Input $\rightarrow 5$, $1 + 2 + 3 + 4 + 5 = 15$ o/p: 15

Recursive thinking:

Base case: $\text{sum}(0) = 0$ So, if $n == 0$:
return 0

Smaller problem: $\text{sum}(5) = 5 + \text{sum}(4)$ Therefore,
 $\text{sum}(n) = n + \text{sum}(n-1)$ & n + answer.

Python Template:

```
def sum_n(n):
```

```
    if n == 0:  
        return 0
```

```
    return n + sum_n(n-1)
```

$\text{sum}(5)$

$= 5 + \text{sum}(4)$

$\text{sum}(4) = 4 + \text{sum}(3)$

$\text{sum}(3) = 3 + \text{sum}(2)$

$\text{sum}(2) = 2 + \text{sum}(1)$

o/p: 15

T.C: $O(n)$

S.C: $O(1)$

Iterative version:

```
def sum_n(n):
```

```
    total = 0
```

```
    for i in range(1, n+1):
```

```
        total += i
```

```
    return total
```

T.C: $O(n)$

S.C: $O(1)$

Factorial

vs Sum:

Problem

Relative

Base

Factorial

$n \times f(n-1)$

$f(0) = 1$

Sum

$n + f(n-1)$

$f(0) = 0$

$f(n)$

↓

current value

+

smaller problems

Same

Structure

④ Reverse a string
problem: Given, i/p: 'hello' o/p: 'olleh' without
using loop solving by recursion method.

Recursion - thinking:

S1: hello $\rightarrow$ Break into smaller problem

S2: hello $\rightarrow$ "h" + "ello" Then, "olle" + "h" = "olleh"

S3: Recursive relation: $\text{reverse}(\text{hello}) = \text{reverse}(\text{ello}) + \text{h}$

S4: General idea: $\text{reverse}(\text{string}) = \text{reverse}(\text{string}[1:]) + \text{string}[0]$

Python Template:

```
def reverse_string(s):
```

```
    if len(s) <= 1:
```

```
        return s
```

```
    return reverse_string(s[1:]) + s[0]
```

```
Print(reverse_string("hello"))
```

o/p: olleh

pattern:

string
 $\downarrow$

Take first character

$\downarrow$

Solve remaining string

$\downarrow$

Put first character at end

Formula:

$$\text{reverse}(s) = \text{reverse}(s[1:]) + s[0]$$

⑤ Power of a Number

Problem: Given two numbers x and n , calculate x^n . Example: $2^5 = 2 \times 2 \times 2 \times 2 \times 2 = 32$

So, I/P: $x=2, n=5$ o/p: 32

Recursive thinking:

Base case: any number power of 0: $x^0 = 1$

So, if $n=0$:
return 1

Smaller problem: $2^5 \Rightarrow 2^5 = 2 \times 2^4, 2^4 = 2 \times 2^3 \dots$

Therefore, $\text{power}(x, n) = x * \text{power}(x, n-1)$

What we do with smaller answers?

Multiply by x : return $x * \text{power}(x, n-1)$

Python Template:

```
def power(x, n):
```

```
    if n == 0:
```

```
        return 1
```

```
    return x * power(x, n-1)
```

```
print(power(2, 5))    o/p: 32
```

$\text{power}(2, 5)$

$$= 2 \times \text{pow}(2, 4)$$

$$= 2 \times 2 \times \text{pow}(2, 3)$$

$$= 2 \times 2 \times 2 \times \text{pow}(2, 2)$$

$$= 2 \times 2 \times 2 \times 2 \times \text{pow}(2, 1)$$

$$= 2 \times 2 \times 2 \times 2 \times 2 \times \text{pow}(2, 0)$$

$$= 32$$